# GABRYEL VERÍSSIMO

Aspiring Backend Software Engineer · Computer Science Student

London, SW9 9ET · +44 7742 057132 · gabryelverissimo12@gmail.com

github.com/gabryelvs | linkedin.com/in/gabryel-veríssimo | portfolio-gabryelverissimo.vercel.app

## PROFILE

Computer Science student (Year 3, University of Greenwich) building production-grade backend systems in Python and FastAPI, with a focus on fintech APIs — payments, ledgers, idempotent transfers, and secure authentication — and shipping fullstack tools with React and TypeScript. Comfortable with PostgreSQL, REST design, Docker, and test-driven development. Seeking a junior or placement backend or fullstack developer role in London.

## TECHNICAL SKILLS

**Languages:** Python, Java, C#, SQL, JavaScript, C++, HTML/CSS

**Backend:** FastAPI, Spring Boot, Spring Security, REST APIs, PostgreSQL, Redis, SQLAlchemy, Alembic, JWT authentication

**Frontend:** React, Next.js, TypeScript, Vite, Tailwind CSS, REST API integration

**Tools & Practices:** Docker, Git, GitHub Actions (CI), pytest, Vitest, test-driven development, background workers & queues, Linux, AI-assisted development (Claude, GitHub Copilot)

## PROJECTS

### PayLedger — Double-Entry Payments API

*Python · FastAPI · PostgreSQL · Redis · Docker | github.com/gabryelvs/payledger | live demo*

- Backend payment platform built around an immutable double-entry ledger: balances and transfers recorded as balanced, append-only ledger entries.
- Race-safe transfers via row-level database locking (no lost updates under concurrent load); idempotent writes; money stored as integer minor units to avoid floating-point errors.
- Test-driven (31 tests incl. a concurrency proof), with Docker Compose, GitHub Actions CI, and Alembic migrations.

### Webhook Inspector — Fullstack Webhook Debugging Tool

*Python · FastAPI · PostgreSQL · React · TypeScript · Tailwind · Docker | github.com/gabryelvs/webhook-inspector | live demo*

- Fullstack tool for debugging webhooks: users create a disposable URL, point any provider at it, and watch requests arrive live in a React interface showing headers, pretty-printed body, and query parameters.
- Hardened for public deployment: per-IP rate limiting, streamed request bodies capped at 1 MB so a large payload cannot exhaust memory, per-bin retention limits, and a capture endpoint that always answers 200 so a database fault never breaks the sender's webhook.
- Test-driven across the stack (25 backend pytest + 7 frontend Vitest tests), single-container Docker build serving the API and compiled React app, deployed on Fly.io with PostgreSQL.

### FX-Service — Async Currency Exchange API

*Python · FastAPI · httpx · Redis | github.com/gabryelvs/fx-service | live demo*

- Async microservice serving live currency rates and conversions; caches ECB rates in Redis with an in-process scheduled refresh loop.
- Resilient: keeps serving last-known rates when the upstream provider is down (stale fallback), proven by an automated outage test.
- Test-driven (31 tests, 90% coverage) with Docker Compose and GitHub Actions CI.

### Webhook-Dispatcher — Reliable Webhook Delivery

*Python · FastAPI · Redis | github.com/gabryelvs/webhook-dispatcher*

- Delivers webhooks via a Redis queue and a separate worker process, signing each request (HMAC-SHA256) so receivers can verify authenticity.
- Retries with exponential backoff and jitter, dead-letters exhausted deliveries, and supports manual replay; at-least-once delivery with de-duplication headers.
- Test-driven (28 tests, 92% coverage) with Docker Compose and GitHub Actions CI.

### Taskboard API — Trello-like Task Manager API

*Java 21 · Spring Boot · PostgreSQL · Docker | github.com/gabryelvs/taskboard-api | live demo*

- Trello-like task manager REST API with JWT authentication using refresh-token rotation and family revocation on reuse detection, and role-based project membership (OWNER/MEMBER) with 404-no-leak authorization.
- Transactional drag-and-drop card ordering with pessimistic column locking (deterministic lock order) proven under concurrent-move integration tests; RFC 7807 problem+json error responses.
- Test-driven (62 Testcontainers integration tests against real PostgreSQL), OpenAPI/Swagger docs, GitHub Actions CI, and deployed on Fly.io.

### SECTOR—9 — Animated Demo Storefront

*TypeScript · Next.js 16 · React 19 · Tailwind 4 · GSAP · Docker | github.com/gabryelvs/store-demo | live demo*

- Storefront demo built as a client-facing sales asset: 20-product catalogue, hover-swap product grid, quick-view size selection and a persistent cart drawer, with the catalogue behind a single data seam so a real backend replaces it without touching the pages.
- Scroll-driven GSAP reveals and a parallax band, with three overlays (quick-view, cart, mobile navigation) that each trap focus, mark background content inert for screen readers, restore focus to their opener, and stand down entirely under prefers-reduced-motion.
- Cart is a pure reducer with derived totals in integer pence and validated localStorage rehydration (30 Vitest tests); 26 statically prerendered routes shipped as a standalone container on Fly.io, scoring 99–100 on Lighthouse performance, accessibility, best practices and SEO with zero layout shift.

### OWASP Security Lab — Break it & Fix it

*Python · FastAPI · pytest · Docker | github.com/gabryelvs/owasp-security-lab*

- Intentionally-vulnerable FastAPI app demonstrating six OWASP Top 10 issues (SQL injection, broken access control, SSRF, JWT auth flaws, sensitive-data exposure, security misconfiguration) — each with a working exploit, a hardened fix, and tests proving both.
- Test-driven (14 exploit/fix tests) with GitHub Actions CI; documents the attack, root cause, and remediation per vulnerability — demonstrating secure-coding and defensive engineering.

*Academic & coursework projects*

- **Secure Web App with Multi-Factor Authentication** (Python) — built and presented a multi-step secure login system with a classmate.
- **Database Design, Security & Optimisation** (SQL) — designed a database from scratch and hardened an existing one by fixing vulnerabilities.
- Other team projects: led a team of four building an e-commerce site (HTML/CSS/JS); a Python + database family-tree system; and an educational mobile game.

## WORK EXPERIENCE

### Web Designer & Media, Impact Brixton — 2021

- Worked within a team to build a new company website (layout, visual content, UX) and supported social media and digital marketing.

## EDUCATION

**BSc Computer Science** — University of Greenwich (2023–Present): Foundation Year, Year 1, Year 2, currently Year 3.

City of Westminster College (2022–23): BTEC Level 2 ICT · GCSE English. Lambeth College (2021–22): L1 Diploma ICT · GCSE Maths · FS Maths & English. SESI-SENAI College (2019–20): Diploma in Microsoft Applications.

## LANGUAGES

**Portuguese:** Native **English:** Fluent (advanced — speaking, reading, writing)